

DEPARTEMENT INFORMATIQUE - IUT 2 GRENOBLE



2024 – 2025 A2

DOCUMENT TECHNIQUE

GESTION DU CYCLE DE VIE DES COMPTE UTILISATEURS

MAIRIE D'ECHIROLLES



Présenté par

Nicolas Tsigris

Jury

IUT : Mme. maignan

IUT : M. Bonnaud

Société : M. Le Brouster

Déclaration de respect des droits d'auteurs

Par la présente, je déclare être le seul auteur de ce rapport et assure qu'aucune autre ressource que celles indiquées n'ont été utilisées pour la réalisation de ce travail. Tout emprunt (citation ou référence) littéral ou non à des documents publiés ou inédits est référencé comme tel et tout usage à un outil doté d'IA a été mentionné et sera de ma responsabilité.

Je suis informé qu'en cas de flagrant délit de fraude, les sanctions prévues dans le règlement des études en cas de fraude aux examens par application du décret 92-657 du 13 juillet 1992 peuvent s'appliquer. Elles seront décidées par la commission disciplinaire de l'UGA.

A Notre dame de commiers, 38450 Le 07/06/2025 Signature :

A handwritten signature in black ink, consisting of several overlapping loops and a long horizontal stroke extending to the right.

Remerciements

Je souhaite exprimer ma sincère gratitude à toutes les personnes qui ont contribué à la réussite de mon stage et à l'élaboration de ce rapport.

Tout d'abord, un immense merci à Monsieur Le Brouster, mon maître de stage à la mairie d'Échirolles, pour son accompagnement attentif, ses précieux conseils et son engagement constant. Son expertise et sa disponibilité ont été des atouts majeurs pour mon apprentissage et la réussite de ce projet.

Je remercie également les membres du jury de l'IUT, pour leur suivi.

Un grand merci à l'équipe informatique de la mairie d'Échirolles qui m'a accueilli avec bienveillance et m'a offert un cadre de travail stimulant. Nos échanges m'ont permis de mieux appréhender les enjeux de la gestion des systèmes d'information en milieu public et d'enrichir mes compétences techniques.

Cette immersion au sein de la mairie d'Échirolles m'a permis de développer des compétences essentielles, d'explorer des solutions innovantes et de mieux comprendre les défis liés à la gestion informatique dans une collectivité. Je garderai un souvenir précieux de cette expérience enrichissante.

Table des matières

I.	INTRODUCTION	6
II.	PRESENTATION DE L'ENTREPRISE ET DU STAGE	7
II.1	PRESENTATION DE LA MAIRIE D'ECHIROLLES.....	7
1.	<i>Rôle et missions.....</i>	7
2.	<i>Taille et moyens humains.....</i>	7
3.	<i>Moyens matériels et infrastructures.....</i>	7
4.	<i>Système d'information et stratégie numérique.....</i>	7
II.2	PRESENTATION DU STAGE.....	8
1.	<i>Sujet du stage et contexte.....</i>	8
2.	<i>Objectifs et livrables.....</i>	9
3.	<i>Planning prévisionnel et avancement réel.....</i>	10
4.	<i>Positionnement par rapport à l'existant.....</i>	10
III.	PRESENTATION DE LA REALISATION	11
III.1	CONTEXTE ET OBJECTIFS TECHNIQUES.....	11
III.2	FONCTIONNEMENT DE LSC.....	11
III.3	DETECTION DES MODIFICATIONS ET STRUCTURATION DES DONNEES.....	12
III.4	INTERFACE DE SUPERVISION ET VALIDATION DES CHANGEMENTS.....	12
III.5	5. RAISONS DES CHOIX TECHNIQUES	13
	<i>Choix du langage de développement.....</i>	13
	<i>Choix du framework web.....</i>	13
	<i>Intérêt de cette architecture.....</i>	14
III.6	DETAIL DES TACHES	14
	<i>Analyse du besoin et rédaction du cahier des charges.....</i>	15
	<i>Recherche et comparaison de solutions techniques.....</i>	15
	<i>Conception de l'architecture logicielle.....</i>	15
	<i>Développement logiciel.....</i>	15
	<i>Tests et validation.....</i>	15
	<i>Documentation.....</i>	15
III.7	DESCRIPTION DES FONCTIONNALITES REALISEES.....	16
	<i>Fonctionnalités principales.....</i>	16
III.8	ARCHITECTURE LOGICIELLE.....	16
III.9	ALGORITHME DE COMPARAISON LDIF/LDAP.....	16
III.10	INTERFACE UTILISATEUR	16
III.11	POINTS TECHNIQUES PARTICULIERS.....	17
IV.	ANNEXES.....	19
IV.1	ANNEXE A – DIAGRAMMES D'ARCHITECTURE.....	19
IV.2	ANNEXE B – EXTRAIT DE FICHIER LDIF	19
IV.3	ANNEXE C – FORMAT DU FICHIER JSON DES COMPTES IGNORES.....	19
IV.4	ANNEXE D – INTERFACE WEB : CAPTURES D'ECRAN.....	19
IV.5	ANNEXE E – EXTRAITS DE CODE	21
IV.6	ANNEXE F – EXEMPLE DE CONFIGURATION.....	21
IV.7	ANNEXE G – REQUETES LDAP	21
IV.8	ANNEXE H – JEUX DE TESTS.....	22
IV.9	ANNEXE I – CONTEXTE ORGANISATIONNEL	22
IV.10	ANNEXE J – FONCTIONNEMENT DE LSC (LDAP SYNCHRONIZATION CONNECTOR)	22

Table des figures

Figure 1 - Logo de L'iut	1
Figure 2 - Logo De la Ville	1
Figure 3 - Diagramme d'architecture	19
Figure 4 - Interface Web pagination	20
Figure 5 - Détails des comptes	20
Figure 6 - Exemple de route Flask	21
Figure 7 - Fichier de conf anonymisé	21
Figure 8 - Exemple de requête LDAP	22
Figure 9 - Format du fichier LDIF	24

I. INTRODUCTION

Dans le cadre de ma formation, j'ai eu l'opportunité de réaliser un stage au sein de la mairie d'Échirolles, intégrant une équipe restreinte composée principalement de mon maître de stage et d'un développeur chargé d'application. **Échirolles** est une commune située dans le département de l'Isère, en région Auvergne-Rhône-Alpes. Elle compte environ 36 000 habitants et dispose d'une administration municipale qui gère divers services publics tels que l'urbanisme, l'état civil, les affaires sociales et la gestion des infrastructures locales.

Ce stage s'inscrit dans une volonté de modernisation et d'optimisation du système d'information de la ville, avec pour objectif principal l'**automatisation du cycle de vie des comptes utilisateurs**. La gestion des comptes utilisateurs constitue un enjeu stratégique pour la ville d'Échirolles. Il est essentiel que la création, la modification et la suppression des comptes puissent être effectuées de manière automatique et cohérente afin d'assurer une administration efficace et sécurisée des accès aux services numériques. L'objectif est de synchroniser ces opérations avec les données issues de la base des ressources humaines, garantissant ainsi une correspondance fiable entre les employés actifs et leur accès aux services tels que la messagerie électronique et le VPN.

Au cours de ce stage, mon rôle a consisté à travailler en **étroite collaboration avec mon maître de stage et un autre membre de l'équipe**, afin de concevoir et développer ces différents éléments. J'ai ainsi participé à la formalisation des processus, à la rédaction des scripts d'automatisation et à l'élaboration d'une interface web facilitant la gestion des comptes utilisateurs.

II. PRESENTATION DE L'ENTREPRISE ET DU STAGE

II.1 Présentation de la mairie d'Échirolles

1. Rôle et missions

La mairie d'Échirolles est une institution publique chargée de l'administration et du développement de la ville. Contrairement à une entreprise qui vend des produits ou services, la mairie offre directement aux habitants une série de prestations essentielles : gestion de l'urbanisme, état civil, action sociale, développement culturel, maintenance des infrastructures et préservation de la sécurité publique. La collectivité se positionne ainsi en garant du cadre de vie local, en cohérence avec une démarche de développement durable et d'innovation.

2. Taille et moyens humains

La mairie d'Échirolles emploie environ 1 000 agents municipaux répartis dans divers services et directions. L'organisation s'articule autour de trois grands pôles :

- **Population** : chargé de l'état civil, de l'organisation des élections et de la promotion de la citoyenneté.
- **Territoire** : responsable de l'urbanisme, de l'aménagement, de l'entretien des espaces publics et de la propreté urbaine.
- **Solidarité et habitat** : axé sur l'action sociale, le logement et l'accompagnement des personnes en difficulté. Chaque filière – administrative, technique, culturelle, sociale et sportive – contribue activement à la réalisation des politiques publiques au service de la population.

Les agents municipaux sont répartis en différentes filières : administrative, technique, culturelle, sociale et sportive. La filière technique est la plus représentée avec grand nombre d'agents.

3. Moyens matériels et infrastructures

Pour assurer ses missions, la mairie met à disposition plusieurs infrastructures matérielles :

- **Hôtel de ville** : situé Place des Cinq Fontaines, il regroupe les principaux services administratifs.
- **Centres sociaux et culturels** : espaces d'accueil et d'accompagnement destinés aux habitants.
- **Équipements sportifs** : gymnases, stades et piscines facilitant l'accès aux activités physiques et de loisirs.
- **Espaces verts** : parcs et jardins entretenus pour favoriser la biodiversité et offrir un cadre de vie agréable.

4. Système d'information et stratégie numérique

Pour mener à bien ses politiques locales, la ville d'Échirolles s'appuie sur un système d'information moderne, articulé autour de deux axes complémentaires :

Solutions numériques libres :

Depuis 2014, la ville a progressivement remplacé ses outils propriétaires par des alternatives open source. Ce choix stratégique, visant à renforcer la souveraineté numérique, permet une meilleure maîtrise des données et une réduction des coûts. Aujourd'hui, Échirolles est reconnue comme une référence nationale en la matière. Parmi les logiciels utilisés figurent LibreOffice, Nextcloud, Zorin OS, qui offrent aux agents municipaux des outils performants et indépendants des solutions propriétaires.

Infrastructure informatique locale :

La mairie gère directement la plupart de ses services informatiques sur place grâce à ses propres serveurs. Cette approche assure une maîtrise complète des données, renforce la sécurité des systèmes et garantit l'indépendance vis-à-vis des prestataires externes. En hébergeant ses services en interne, Échirolles maintient la cohérence de sa politique de souveraineté numérique tout en optimisant les coûts opérationnels.

La mairie d'Échirolles s'efforce d'optimiser ses services et de répondre aux besoins des citoyens en mettant en place des projets innovants et en modernisant son système d'information.

II.2 Présentation du stage

1. Sujet du stage et contexte

Dans le cadre de la modernisation du Système d'Information de la mairie d'Échirolles, mon stage porte sur l'automatisation du cycle de vie des comptes utilisateurs. L'objectif principal est d'assurer une gestion fluide et cohérente des comptes en les synchronisant automatiquement avec la base de données des ressources humaines, garantissant ainsi une meilleure efficacité et une réduction des interventions manuelles.

II.2.5.1 Origine du besoin

À l'origine du projet, la mairie souhaitait fournir une adresse mail professionnelle à l'ensemble de ses agents afin de standardiser les échanges internes et faciliter la communication entre les différents services. Cependant, la mise à disposition de cette adresse nécessite la création et la gestion d'un compte utilisateur dans le Système d'Information de la ville. Or, ce processus était jusque-là entièrement manuel, impliquant une charge de travail conséquente pour les administrateurs du système et introduisant un risque d'erreurs ou d'incohérences dans la gestion des accès.

II.2.5.2 Les défis du système actuel

Jusqu'à présent, l'ajout, la modification et la suppression des comptes utilisateurs étaient réalisés de manière manuelle à partir d'un tableau de suivi et de demandes spécifiques transmises par les gestionnaires RH. Ce fonctionnement entraînait plusieurs limites :

- **Lourdeur administrative** : chaque création ou suppression de compte devait être traitée individuellement, ce qui mobilisait les équipes informatiques et ralentissait le processus.
- **Risque d'incohérences** : l'absence de synchronisation avec la base RH pouvait entraîner des erreurs dans les droits d'accès et des oublis lors des départs ou arrivées d'agents.

Pour pallier ces problèmes, la mairie a souhaité mettre en place un système d'automatisation basé sur la synchronisation avec la base de données des ressources humaines. Ce système permettra de **créer un compte automatiquement dès qu'un agent est ajouté à la base RH**, et de **le supprimer lorsqu'il quitte la collectivité**, garantissant ainsi une cohérence entre les informations du personnel et les accès numériques.

II.2.5.3 Solutions envisagées

Afin de répondre à ces besoins, plusieurs éléments techniques ont été intégrés au projet :

- **Synchronisation des données RH avec l'Active Directory via LDAP Synchronization Connector (LSC)** : cette solution permet de lier directement les informations du personnel aux comptes informatiques, évitant toute manipulation manuelle.
- **Automatisation des opérations** : des scripts ont été développés pour configurer et modifier les comptes utilisateurs sans intervention humaine.
- **Contrôle et vérification** : un module de gestion des incohérences a été ajouté pour garantir l'intégrité des données entre l'annuaire et les services numériques.
- **Interface web de supervision** : un tableau de bord accessible aux administrateurs leur permet de suivre l'évolution des synchronisations et d'exécuter des actions spécifiques si nécessaire.

Grâce à cette approche, la mairie d'Échirolles pourra fluidifier la gestion des comptes utilisateurs, améliorer la fiabilité des accès et réduire la charge de travail des équipes informatiques tout en garantissant une meilleure maîtrise des données personnelles des agents.

2. Objectifs et livrables

Les objectifs du stage sont les suivants :

1. Analyser les besoins en collaboration avec mon maître de stage et un autre collaborateur.
2. Formaliser schématiquement les processus de création, modification et suppression des comptes utilisateurs.
3. Développer des scripts pour automatiser ces opérations sur les différentes plateformes concernées.
4. Mettre en place une procédure de vérification garantissant l'intégrité des données entre l'Active Directory et les services.
5. Proposer une première version de l'interface web pour le suivi des opérations et la gestion manuelle des synchronisations.
6. Évaluer la possibilité d'étendre ce système à d'autres services comme la téléphonie fixe (Xivo).

Les livrables produits pendant le stage incluent :

- **Une documentation détaillée** sur le fonctionnement du système et les scripts d'automatisation.

- **Un ensemble de scripts** permettant la gestion automatique des comptes utilisateurs.
- **Une analyse des incohérences** détectées dans les comptes existants avant la mise en œuvre du système.
- **Une interface web** permettant de visualiser l'état des synchronisations et d'exécuter des actions.

3. Planning prévisionnel et avancement réel

Voici le planning prévisionnel établi au début du stage ainsi que l'état d'avancement actuel :

Semaine	Tâches prévues	État d'avancement
Semaine 1	Prise en main de LSC, de la VM et de la documentation	✓ Terminée
Semaine 2	Configuration de base de LSC (source → cible LDAP), tests et génération de fichiers LDIF	✓ Terminée
Semaine 3	Finalisation de la config LSC, gestion des erreurs et tests avec jeu de données	✓ Terminée
Semaine 4	Ajustements avancés de la configuration, vérifications de stabilité et préparation pour l'intégration web	✓ Terminée
Semaine 5/6	Analyse des fichiers LDIF, création d'une interface web pour visualiser les modifications, mise en place de notifications	✓ Terminée
Semaine 7/8	Ajout des actions "Appliquer / Ignorer", exécution des modifications via scripts/API, journalisation des actions	✓ Terminée

Bien que le planning ait été bien structuré au départ, certaines étapes ont pris plus de temps que prévu, notamment sur la partie **interface web et gestion des modifications** (semaines 5 et 6). Cela s'explique par des ajustements nécessaires et des défis techniques rencontrés.

4. Positionnement par rapport à l'existant

Avant le début du stage, l'**Active Directory** jouait le rôle de **source de vérité RH**, contenant les informations de référence sur les agents (identité, service, statut, etc.). Cependant, **il n'existait pas de mécanisme automatisé** permettant de synchroniser ces données avec l'**annuaire informatique utilisé pour gérer les comptes utilisateurs** au sein du système d'information de la ville. La mise à jour de cet annuaire se faisait donc manuellement, ce qui pouvait entraîner des erreurs ou des retards dans la gestion des comptes.

Mon travail s'est appuyé sur cet existant pour **automatiser le processus de mise à jour**, en mettant en place une solution capable de détecter les écarts entre l'Active Directory et l'annuaire informatique, et de proposer une validation fine des modifications à appliquer.

L'analyse de la situation initiale a conduit à la **rédaction d'un cahier des charges**, joint en annexe de ce rapport. Ce document détaille les besoins identifiés et les améliorations nécessaires pour garantir une intégration fiable, automatisée et contrôlée entre les deux systèmes.

III. PRESENTATION DE LA REALISATION

III.1 Contexte et objectifs techniques

L'objectif principal du stage était de mettre en place un mécanisme d'automatisation pour la gestion des comptes utilisateurs de la mairie d'Échirolles, en s'appuyant sur les données issues des ressources humaines. Il s'agissait de concevoir un système capable de détecter les différences entre les données RH et l'annuaire Active Directory, tout en permettant une validation manuelle des modifications avant leur application. Ce projet visait donc à automatiser tout en conservant un contrôle humain sur les opérations sensibles, afin d'éviter les erreurs, de renforcer la sécurité et de garantir la traçabilité.

III.2 Fonctionnement de LSC

LSC (LDAP Synchronization Connector) est un outil open source développé en Java, utilisé pour synchroniser des données entre deux annuaires LDAP. Dans le cadre de ce projet, **la source** est l'**Active Directory** (où sont créés les comptes à partir des données RH), et **la cible** est l'**annuaire informatique de la ville**, qui contient les comptes réellement utilisés dans les systèmes informatiques.

LSC compare régulièrement les deux systèmes. Lorsqu'il détecte un écart entre la source (AD) et la cible (annuaire interne), il génère un fichier **LDIF** contenant les modifications nécessaires pour que la cible soit mise à jour et reste cohérente avec la source.

Ce fichier LDIF contient les opérations à effectuer : ajouts de comptes, suppressions ou modifications d'attributs (nom, service, adresse électronique, etc.).

Cependant, dans ce projet, **LSC n'applique pas automatiquement ces changements**. Pour éviter les erreurs ou les mises à jour non souhaitées, une vérification manuelle est mise en place avant toute application effective.

III.3 Détection des modifications et structuration des données

Pour offrir un contrôle humain sur les changements proposés par LSC, une **surcouche logicielle en Python** a été développée. Elle permet de **parser**, **structurer** et **filtrer** les fichiers LDIF générés, afin de ne retenir que les modifications validées par l'administrateur via une interface web.

Deux scripts principaux assurent ce traitement :

a. LDIFParserDiff : lecture et structuration des modifications

Le script `LDIFParserDiff` lit le fichier LDIF produit par LSC et identifie toutes les opérations qu'il contient (ajout, suppression, modification). Ces opérations sont ensuite **regroupées par utilisateur**, grâce à leur **DN** (Distinguished Name), ce qui permet de visualiser clairement les modifications proposées pour chaque compte.

Cette organisation est essentielle pour permettre à l'administrateur de comprendre, compte par compte, quelles actions sont prévues.

b. LDIFParserApply : filtrage et génération d'un fichier LDIF propre

Une fois les changements validés dans l'interface, le script `LDIFParserApply` prend en compte ces choix et génère un **fichier LDIF filtré**, contenant uniquement les modifications approuvées.

III.4 Interface de supervision et validation des changements

Une interface web a été conçue pour permettre aux administrateurs de visualiser, valider ou ignorer les changements. Elle a été développée avec le micro-framework **Flask**, qui permet de mettre en place rapidement une application web en Python. Le rendu HTML repose sur **Bootstrap**, un framework CSS permettant de proposer une interface responsive, claire et moderne.

L'interface présente les changements par utilisateur, en listant les attributs modifiés, leurs anciennes valeurs et leurs nouvelles. Pour chaque modification, l'utilisateur peut choisir de **l'appliquer** ou de **l'ignorer**. Une fois les choix validés, les modifications sélectionnées sont traitées par le script `LDIFParserApply`.

Ce système donne un contrôle total sur les opérations appliquées à l'Active Directory, tout en gardant une forte traçabilité : toutes les actions sont journalisées, et un historique peut être conservé pour audit ou consultation.

III.5 5. Raisons des choix techniques

Choix du langage de développement

Le développement du système de traitement et d'application des différences LDIF aurait pu être envisagé en PHP, un langage que je maîtrise déjà, notamment utilisé pour les applications web. Toutefois, une étude comparative des bibliothèques disponibles a révélé plusieurs limites significatives à ce choix.

En PHP, plusieurs bibliothèques ont été explorées :

- **php-ldif-parser** se limite au traitement de fichiers LDIF, avec des fonctionnalités très réduites.
- **ldaptools/ldaptools** propose une couverture plus large, incluant LDAP et LDIF, mais reste complexe à mettre en œuvre et manque de documentation claire.
- **Adldap2** est performant pour interagir avec un Active Directory via LDAP, mais ne prend pas en charge les fichiers LDIF, ce qui est essentiel dans notre cas.

En comparaison, Python dispose d'outils beaucoup plus adaptés au besoin :

- **python-ldap** est une bibliothèque mature, bien documentée, avec un excellent support LDAP.
- Le module **ldif**, intégré à python-ldap, permet de lire et écrire des fichiers LDIF de façon simple et efficace.

En plus de la richesse des bibliothèques, Python présente de nombreux avantages structurels :

- Il est conçu pour le scripting, l'automatisation et le traitement de fichiers.
- Il facilite le développement en ligne de commande, avec une gestion simplifiée des entrées/sorties.
- C'est un langage polyvalent, très répandu, et pertinent à apprendre pour des projets futurs.

En résumé, **le choix de Python s'est imposé naturellement** : il est plus adapté à la manipulation de fichiers LDIF, plus ergonomique pour les scripts, et son écosystème propose des outils robustes et bien documentés. De plus, ce choix m'a permis de renforcer mes compétences dans un langage moderne et très utilisé dans le milieu professionnel.

Choix du framework web

Une fois le traitement des données automatisé, il fallait concevoir une interface permettant de valider manuellement les changements détectés avant de les appliquer. Le choix du framework web devait répondre à plusieurs critères : légèreté, simplicité de mise en œuvre, et capacité à générer du HTML sans complexité inutile.

Plusieurs frameworks ont été évalués :

- **Django** est très complet (ORM, gestion des utilisateurs, templates), mais il s'avère trop structurant pour un projet de cette taille, avec une configuration initiale relativement lourde.
- **FastAPI**, très performant pour des APIs REST et les applications asynchrones, n'est pas optimisé pour le rendu HTML.
- **Tornado** et **Pyramid** sont puissants, mais davantage adaptés à des projets complexes, voire à des applications temps réel.
- Finalement, **Flask** s'est révélé être la meilleure option.
- Flask est un micro-framework web en Python qui permet de créer des applications web légères et flexibles. Il fournit les composants essentiels pour développer un site ou une API, tels que la gestion des routes (URL), des requêtes HTTP, et des sessions. Contrairement à des frameworks plus complets comme Django, Flask laisse à l'utilisateur le choix des outils complémentaires (authentification, base de données, etc.), ce qui le rend très adapté aux projets personnalisés ou de petite taille. Il est simple à prendre en main, bien documenté, et largement utilisé dans la communauté Python.
- Jinja2, souvent utilisé avec Flask, est le moteur de templates intégré. Il permet de générer dynamiquement des pages HTML à partir de variables Python et de structures de contrôle comme des boucles ou des conditions. L'association de Flask et Jinja2 rend possible la construction rapide d'interfaces web interactives, où la logique Python côté serveur peut être liée de façon claire et lisible à l'affichage côté utilisateur. Dans ce projet, Flask a permis de créer l'interface web permettant de naviguer dans les différences entre l'annuaire LDAP et les fichiers LDIF, et Jinja2 a servi à structurer les pages HTML.

En conclusion, **le duo Python + Flask** s'est imposé pour son efficacité, sa souplesse et sa bonne adéquation au projet : traitement en ligne de commande, génération dynamique des modifications, et affichage clair à travers une interface web personnalisée.

Intérêt de cette architecture

Cette architecture permet de combiner :

- **La puissance de LSC** pour détecter automatiquement les écarts entre la source (Active Directory) et la cible (annuaire interne).
- **La souplesse de Python** pour structurer les modifications, les présenter dans une interface web, et générer un fichier de mise à jour conforme aux choix de l'administrateur.
- **La sécurité opérationnelle**, en introduisant une étape de validation humaine avant toute action sur les comptes utilisateurs.

Cette approche assure une **gestion rigoureuse et sécurisée des comptes informatiques**, tout en limitant les erreurs ou les actions non désirées sur l'annuaire de production.

III.6 Détail des tâches

Mon stage s'est articulé autour de plusieurs **étapes clés**, allant de l'analyse des besoins jusqu'au développement et à la documentation du projet.

Analyse du besoin et rédaction du cahier des charges

Dans un premier temps, j'ai pris connaissance du contexte métier en lien avec la gestion des comptes utilisateurs dans l'infrastructure informatique de la ville. J'ai échangé avec les utilisateurs et les administrateurs systèmes pour **recueillir les besoins fonctionnels**. À partir de ces éléments, j'ai rédigé un **cahier des charges** décrivant les objectifs, les contraintes techniques et les fonctionnalités attendues. *(travail effectué par moi-même)*

Recherche et comparaison de solutions techniques

J'ai effectué une **étude comparative des bibliothèques existantes** en PHP et Python pour le traitement des fichiers LDIF et l'interaction avec LDAP. À la suite de cette analyse, j'ai proposé et justifié le choix de la **stack technique** :

- **Python** pour sa robustesse en scripting et son bon support LDAP.
- **Flask** pour développer une interface web simple et flexible.
- **ldap3** pour communiquer avec l'annuaire LDAP. *(travail effectué par moi-même)*

Conception de l'architecture logicielle

J'ai ensuite conçu l'architecture logicielle du projet. J'ai **découpé l'application en modules indépendants** : gestion des routes web, parsing LDIF, accès LDAP, et gestion de la configuration. J'ai également produit des **diagrammes d'architecture** pour documenter cette organisation. *(travail effectué par moi-même)*

Développement logiciel

J'ai développé l'ensemble des fonctionnalités principales du projet :

- lecture et analyse des fichiers LDIF générés par LSC,
- comparaison avec les données présentes en temps réel dans l'annuaire LDAP,
- génération d'une vue web claire des différences détectées.

Tests et validation

Des **tests fonctionnels** ont été réalisés à l'aide de jeux de données simulées, afin de vérifier le bon comportement des scripts, la robustesse de l'interface, et la détection fiable des changements. Plusieurs **bugs et cas limites** ont été identifiés et corrigés au fil du développement. *(travail effectué par moi-même)*

Documentation

J'ai rédigé une **documentation technique** pour permettre la reprise ou l'évolution future du projet, ainsi qu'un guide utilisateur pour l'exploitation quotidienne de l'outil. *(travail effectué par moi-même)*

III.7 Description des fonctionnalités réalisées

Fonctionnalités principales

L'application développée permet de :

- **Charger un fichier LDIF** généré par LSC.
- **Comparer son contenu** avec les données réelles présentes dans l'annuaire LDAP.
- **Détecter et afficher** les créations, suppressions et modifications de comptes.
- **Afficher les différences de manière détaillée** (avec les attributs modifiés, et les anciennes/nouvelles valeurs).
- **Gérer une liste de comptes à ignorer**, persistée dans un fichier JSON.
- **Naviguer dans les résultats** grâce à une interface web paginée.

III.8 Architecture logicielle

Le projet est structuré autour de **modules Python spécialisés** :

- `routes.py` : définit les routes Flask, gère les requêtes HTTP, orchestre la logique applicative.
- `ldap.py` : gère la connexion à l'annuaire LDAP et les opérations de lecture.
- `ldifparser.py` : lit et traite les fichiers LDIF pour extraire les changements.
- `config.py` : centralise la configuration de l'application.

Cette architecture modulaire permet de **séparer clairement les responsabilités** de chaque composant, et facilite la maintenance.

Un **schéma d'architecture** (non inclus ici) illustre cette organisation.

III.9 Algorithme de comparaison LDIF/LDAP

L'algorithme de comparaison repose sur la **parsing linéaire du fichier LDIF**. Pour chaque entrée, il détermine le type de changement à effectuer :

- Si le **changetype** est **add** → l'algorithme vérifie que le compte n'est pas encore présent dans LDAP.
- Si le **changetype** est **delete** → il vérifie que le compte existe bien dans LDAP.
- Si le **changetype** est **modify** → il compare les attributs à modifier avec ceux présents dans le LDIF, puis si l'utilisateur choisit d'appliquer ces modifications elles sont appliquées.

III.10 Interface utilisateur

L'interface web, développée avec **Flask** et **Jinja2**, permet :

- De **visualiser la liste paginée** des comptes concernés par une modification.

- D'**inspecter les détails** pour chaque compte : attributs modifiés, différences avant/après.
- De **marquer certains comptes comme ignorés**, et de sauvegarder cette sélection.
- D'**appliquer uniquement les modifications validées**, en générant un fichier LDIF filtré.

Une **capture d'écran de l'interface** (à insérer dans le rapport) montre la présentation claire et structurée des changements, avec la possibilité d'accepter ou d'ignorer chaque modification.

III.11 Points techniques particuliers

- Utilisation de **ldap3**, une bibliothèque Python moderne, pour interagir avec l'annuaire LDAP en lecture sécurisée.
- **Parsing avancé** du fichier LDIF pour supporter les trois types d'opérations LDAP (**add, delete, modify**).
- Gestion des cas limites, tels que les comptes inexistants, les attributs absents ou incohérents.
- Organisation du code en **modules indépendants**, facilitant les évolutions et les tests.

Conclusion

Ce stage au sein de la mairie d'Échirolles m'a permis de mettre en pratique mes compétences en développement et en gestion des systèmes d'information dans un contexte professionnel concret. L'automatisation du cycle de vie des comptes utilisateurs, via l'intégration de LDAP Synchronization Connector (LSC) et le développement d'une interface de supervision, a apporté une solution efficace aux défis rencontrés par l'équipe informatique.

Grâce à cette expérience, j'ai pu approfondir mes connaissances en Python, LDAP, LDIF et Flask, tout en développant une approche méthodique pour analyser les besoins, concevoir une architecture logicielle et assurer la fiabilité des données. Ce projet contribue à une meilleure gestion des accès numériques, réduisant les interventions manuelles et garantissant une synchronisation cohérente avec les ressources humaines.

Enfin, ce stage m'a offert une immersion enrichissante dans le fonctionnement d'une administration publique et m'a permis de mieux comprendre les enjeux liés à la souveraineté numérique et à la sécurité des systèmes d'information. Les compétences acquises seront précieuses pour mes futurs projets et mon évolution professionnelle.

Résumé :

Dans le cadre de ma formation à l'IUT de Grenoble, j'ai effectué un stage à la mairie d'Échirolles, portant sur l'automatisation du cycle de vie des comptes utilisateurs. Le projet visait à optimiser la gestion des accès numériques en synchronisant ces comptes avec les données RH via LDAP Synchronization Connector (LSC). La solution mise en place comprend un module de vérification des incohérences, une interface web pour superviser les modifications et une automatisation des processus à travers des scripts Python. Mon travail a consisté à analyser les besoins, concevoir l'architecture logicielle, développer les fonctionnalités et assurer la validation des données avant leur application. Grâce à ce projet, la mairie bénéficie d'une gestion plus fiable et sécurisée des accès informatiques, réduisant la charge administrative et les erreurs potentielles.

Mots clés : Automatisation, LDAP Synchronization Connector, gestion des comptes utilisateurs, Python, mairie d'Échirolles.

Abstract:

To ensure secure and efficient user account management, organizations must transition from manual processes to automated solutions—this project embodies that shift at Échirolles City Hall.

As part of my studies at IUT Grenoble, I completed an internship at the Échirolles City Hall focusing on the automation of user account lifecycle management. The project aimed to optimize access management by synchronizing accounts with HR data using LDAP Synchronization Connector (LSC). The implemented solution includes an inconsistency detection module, a web interface for supervision, and automated processes via Python scripts. My work involved needs analysis, software architecture design, feature development, and data validation before execution. This system enhances reliability and security in access management, minimizing administrative workload and potential errors.

Keywords : Automation, LDAP Synchronization Connector, user account management, Python, Échirolles City Hall.

IV. Annexes

IV.1 Annexe A – Diagrammes d'architecture

Cette annexe présente l'architecture générale de l'application. Elle permet de visualiser les interactions entre les différents composants techniques, notamment les modules développés en Python/Flask, l'annuaire, et l'outil LSC qui assure l'extraction régulière des données depuis l'Active Directory.

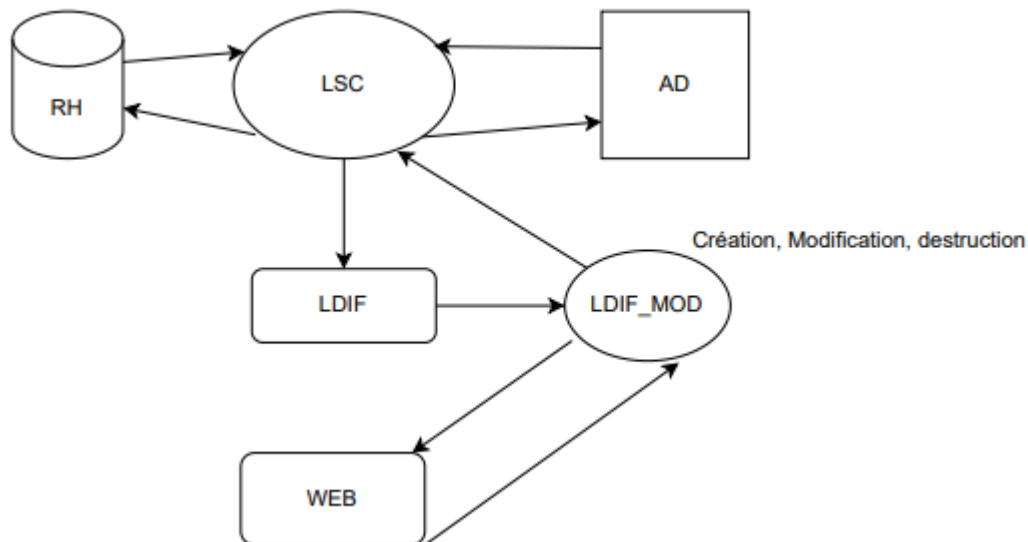


Figure 3 - Diagramme d'architecture

IV.2 Annexe B – Extrait de fichier LDIF

ELABORER UN PEU CA ET DEFINIR MIEU CE QUE C'EST UN LDIF

Cette annexe contient des extraits représentatifs de fichiers LDIF générés automatiquement par LSC. Ils montrent des cas réels de modifications d'attributs sur des comptes utilisateurs (changetype: modify), avec leurs DN et les valeurs remplacées.

IV.3 Annexe C – Format du fichier JSON des comptes ignorés

Pour offrir plus de flexibilité, une liste de comptes à ignorer est maintenue dans un fichier JSON. Cela permet d'exclure manuellement certains comptes du processus de mise à jour automatique, tout en gardant une trace horodatée.

IV.4 Annexe D – Interface web : captures d'écran

L'interface web permet de superviser facilement les écarts entre l'annuaire et les fichiers LDIF. Une liste paginée regroupe les comptes concernés. L'utilisateur peut visualiser les modifications prévues et choisir de les appliquer ou de les ignorer.

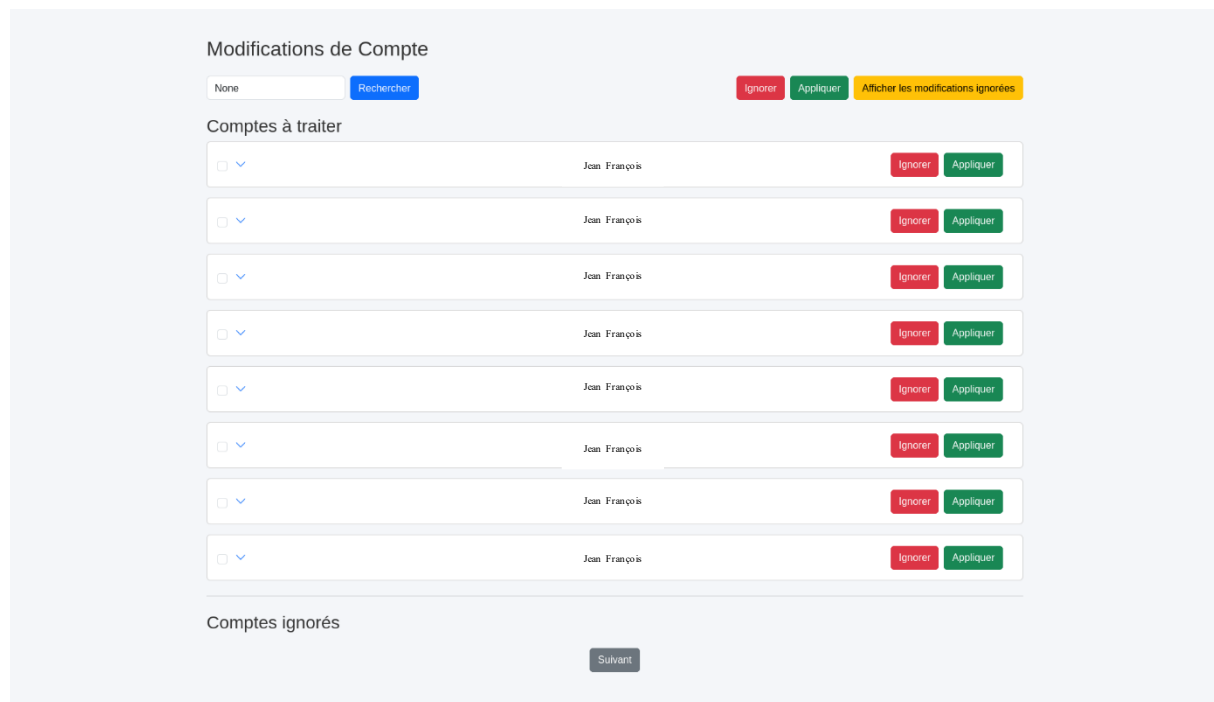


Figure 4 - Interface Web pagination

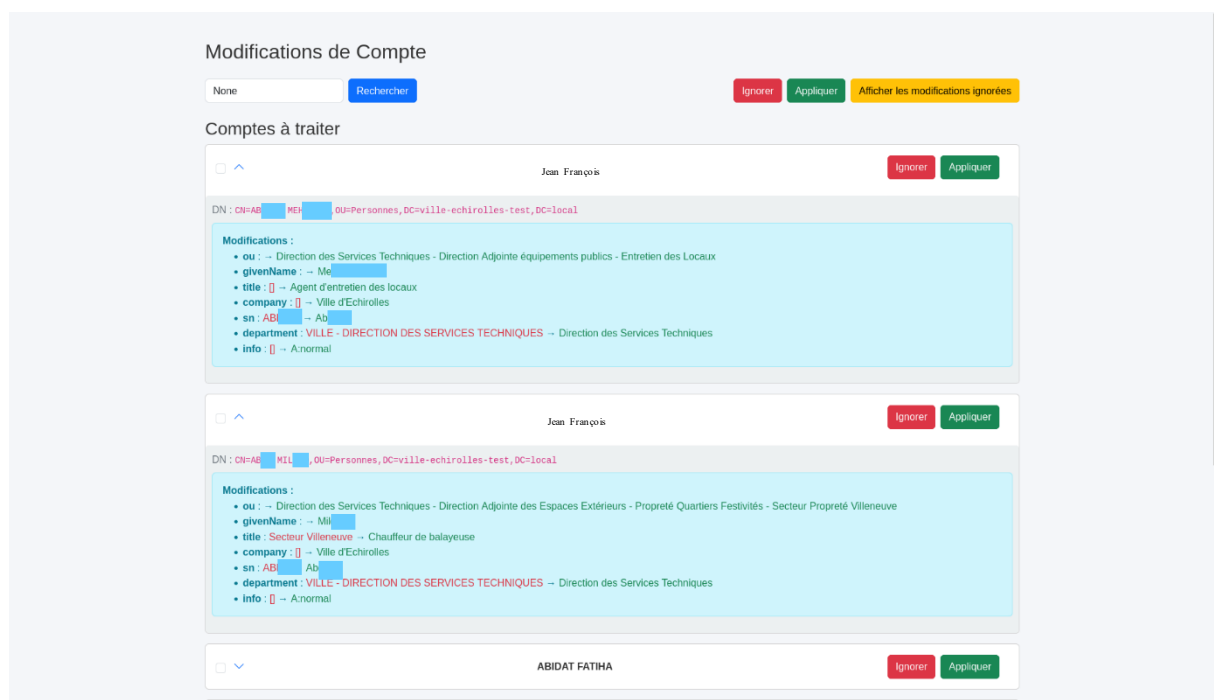


Figure 5 - Détails des comptes

IV.5 Annexe E – Extraits de code

Le projet repose sur deux scripts principaux pour le traitement des LDIF. L'un extrait les différences à afficher, l'autre applique les changements validés. Un extrait représentatif de chaque script est présenté ci-dessous, avec commentaires.

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Bienvenue sur mon API Flask !"

if __name__ == '__main__':
    app.run(debug=True)
```

Figure 6 - Exemple de route Flask

IV.6 Annexe F – Exemple de configuration

Les paramètres essentiels du projet (identifiants LDAP) sont centralisés dans un fichier de configuration. Cela facilite le déploiement et la maintenance de l'application.

```
ldap_uri: 'ldaps://ldap.example.com' # FIXME
domain: XXXXXXXXX
bind_user: app.lsc
bind_pass: 'password for app.lsc' # FIXME
base_dn: 'dc=XXXXXXXXX,dc=local'
person_search_path: 'ou=Personnes,dc=XXXXXXXXX,dc=local'
```

Figure 7 - Fichier de conf anonymisé

IV.7 Annexe G – Requêtes LDAP

Les opérations de lecture et d'écriture sur l'annuaire LDAP sont réalisées à l'aide de commandes standard. Ces requêtes sont appelées automatiquement via les scripts Python.

```
from monmodule.ldap_module import connect, LdapPerson # Remplace par ton chemin réel

def main():
    # Connexion LDAP
    ldap_conn = connect()

    # Nom d'utilisateur à chercher
    username = "jdupont"

    # Recherche de la personne par username
    person = LdapPerson.by_username(ldap_conn, username)

    if person is None:
        print(f"Utilisateur {username} non trouvé dans l'annuaire.")
    else:
        print(f"Nom complet de {username} : {person.displayname}")

if __name__ == "__main__":
    main()
```

Figure 8 - Exemple de requête LDAP

IV.8 Annexe H – Jeux de tests

Pour valider le bon fonctionnement du traitement des fichiers LDIF, des cas de test simulés ont été préparés. Chaque scénario permet de vérifier le comportement de l'application selon différents types de modifications.

IV.9 Annexe I – Contexte organisationnel

L'annuaire informatique centralise les comptes des agents de la mairie, utilisés dans de nombreux services et outils (accès réseau, logiciels métier, messagerie, etc.). Une gestion rigoureuse et automatisée permet d'assurer la fiabilité des données et d'éviter les erreurs manuelles. L'Active Directory, utilisé comme base RH, constitue la source de vérité pour cette synchronisation.

IV.10 Annexe J – Fonctionnement de LSC (LDAP Synchronization Connector)

Présentation générale

LSC (LDAP Synchronization Connector) est un outil open source écrit en Java, utilisé pour synchroniser des données entre une **source** (par exemple un annuaire Active Directory) et une **cible** (comme un annuaire LDAP interne).

Dans le cadre de ce projet, LSC permet d'extraire automatiquement des informations à jour depuis l'Active Directory RH de la mairie (la source), pour générer un fichier LDIF contenant les modifications à appliquer à l'**annuaire informatique interne** (la cible), qui regroupe tous les comptes utilisés par les services municipaux (accès aux logiciels, réseau, messagerie...).

Architecture et rôle de LSC

LSC agit comme un **pont entre deux mondes** :

- **Source: Active Directory**
Contient les informations RH des agents (nom, fonction, service, etc.).
- **Cible : annuaire informatique LDAP**
Utilisé pour l'authentification et l'accès aux ressources métiers.

Plutôt que d'écrire directement dans l'annuaire cible, LSC est configuré pour **générer un fichier LDIF** listant les ajouts, suppressions et modifications nécessaires. Ces opérations sont ensuite contrôlées via une interface web avant d'être appliquées.

Données pivot

Pour faire le lien entre un compte dans la source et un compte dans la cible, LSC utilise une **clé unique appelée "pivot"**.

Ce pivot permet :

- d'identifier un compte de manière stable (même si son nom change),
- de détecter les suppressions (présent dans la cible mais absent de la source).

Le pivot utilisé peut être le `sAMAccountName` ou un identifiant contenu dans le DN (Distinguished Name).

Cycle de synchronisation avec LSC

À chaque exécution, LSC suit les étapes suivantes :

1. **Lecture des comptes dans l'Active Directory (source)**
2. **Lecture des comptes dans l'annuaire informatique (cible)**
3. **Comparaison via le pivot**
 - a. Si le compte existe en source mais pas en cible → création
 - b. S'il existe dans les deux → comparaison attribut par attribut
 - c. S'il existe en cible mais plus en source → suppression
4. **Génération d'un fichier LDIF**
 - a. Contient les opérations `add`, `modify`, ou `delete`
 - b. Utilisé comme entrée pour le traitement Python et l'interface web

Principe général

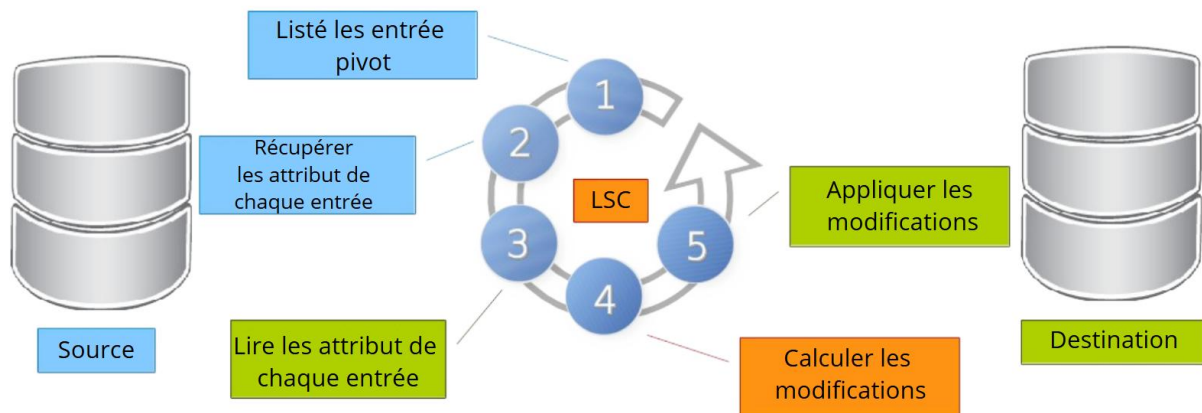


Figure 9 - Workflow LSC

Avantages de cette approche

- **Séparation des responsabilités** : LSC détecte les écarts, mais ne les applique pas directement.
- **Validation humaine** : Le fichier LDIF peut être analysé via une interface web avant toute modification.
- **Synchronisation fiable** : Les données RH sont reflétées automatiquement dans l'annuaire technique, sans intervention manuelle.
- **Historique et traçabilité** : Chaque exécution produit un fichier exploitable.

Exemple simplifié de LDIF et anonymisé généré

```
Wed Apr 16 17:02:41 CEST 2025
dn: CN= ,OU=Personnes,DC=ville-echirolles-test,DC=local
changetype: modify
replace: ou
ou:: RGlyZMN8aW9uXJ2aWNIcyBUZMNobm1xdWVzIC0gRGlyZWNN9uIEFkam9pbnRIIGRlcyB8fc3BhY2VzIEV4dM0pcmlldXJzIpIFF1YXJ0aWVycyB6ZlN8aXZpdM0pcyAtIFNlY3RldXlGUHJvcHJldM0pIFZpbGxlbWV1dmU=
-
replace: givenName
givenName:
-
replace: title
title: Chauffeur de balayeuse
-
replace: company
company: Ville d'Echirolles
-
replace: sn
sn:
-
replace: department
department: Direction des Services Techniques
-
replace: info
info: A:normal
```

Figure 10 - Format du fichier LDIF

Ce type de bloc est ensuite interprété par le parseur Python pour affichage dans l'interface, permettant à un agent de valider ou d'ignorer chaque modification.

Conclusion

LSC constitue le cœur de la synchronisation entre les données RH de l'Active Directory et les comptes techniques utilisés par les services. Il permet une automatisation partielle sécurisée, tout en conservant un contrôle humain final sur les modifications sensibles.

IV.11 Références bibliographiques et webographiques

Bien sûr ! Voici la sitographie réorganisée selon une logique de consultation et par thème :

[1] Stéphane Robert. Développement avec Visual Studio Code et DevContainers [en ligne]. Disponible sur : <https://blog.stephane-robert.info/docs/developper/autres-outils/ide/visual-studio-code/devcontainers/>.

[2] Flask. Configuration Flask [en ligne]. Disponible sur : <https://flask.palletsprojects.com/en/stable/config/>

[3] Flask. Interface en ligne de commande (CLI) [en ligne]. Disponible sur : <https://flask.palletsprojects.com/en/stable/cli/>

[4] Documentation Python. Module venv [en ligne]. Disponible sur : <https://docs.python.org/3/library/venv.html>

[5] LSC Project. Documentation officielle [en ligne]. Disponible sur : <https://www.lsc-project.org/documentation/latest/>

[6] LSC Project. Service Destination Google Apps [en ligne]. Disponible sur : https://www.lsc-project.org/documentation/latest/service_destination_googleapps.html

[7] LSC Project. Exemple de configuration LSC [en ligne]. Disponible sur : <https://github.com/lsc-project/lsc/blob/master/sample/hsqldb/etc/lsc.xml>

[8] LTB Project. Service Desk [en ligne]. Disponible sur : <https://ltb-project.org/documentation/service-desk.html>